

AE646 — Interim Report (Stage 2)

Operator Learning for Parametric Darcy Flow: FNO vs. MLP Baseline

Team PINNacles
Aryamann Srivastava
Department of Aerospace Engineering, IIT Kanpur

September 2026

Contents

1	Introduction and Motivation	2
2	Dataset Setup	2
2.1	Source	2
2.2	Preprocessing	2
2.3	Why Only 900 Validation and 200 Test?	2
3	Model Architectures	2
3.1	MLP Baseline	2
3.2	Fourier Neural Operator (FNO)	3
4	Training Protocol	3
4.1	FNO Improved Configuration	3
5	Preliminary Quantitative Results	3
6	Zero-Shot Super-Resolution	4
7	Inference Speed	5
8	Code Quality and Reproducibility	6
9	Issues Encountered and Next Steps	6
9.1	Issues During Stage 1–2	6
9.2	Next Steps (Stage 3)	6

1 Introduction and Motivation

Traditional numerical solvers such as Finite Difference Methods (FDM) and Finite Element Methods (FEM) require re-solving the full PDE for every new set of parameters, imposing prohibitive computational cost when thousands of queries are needed (e.g., uncertainty quantification, inverse design, real-time control). Neural operators learn a mapping between infinite-dimensional function spaces and amortise this cost over many queries.

This project evaluates two architectures for the 2D parametric Darcy flow problem:

- **MLP baseline** — a fully-connected network with no structural inductive bias.
- **Fourier Neural Operator (FNO)** [1] — a resolution-invariant architecture whose trainable weights reside in the frequency domain.

Both are trained to map the permeability field $\kappa(x, y)$ and spatial coordinates (x, y) to the pressure field $u(x, y)$ satisfying the 2D Darcy equation:

$$-\nabla \cdot (\kappa(x, y) \nabla u(x, y)) = f(x, y), \quad (x, y) \in [0, 1]^2, \quad (1)$$

with homogeneous Dirichlet boundary conditions. The forcing $f = 1$ is constant; the permeability $\kappa \in \{0.1, 1.0\}$ is piecewise-constant, randomly distributed on a 64×64 grid.

2 Dataset Setup

2.1 Source

We use the PDEBench dataset [2] (DOI: 10.18419/darus-2986), a benchmark suite for scientific machine learning. The 2D Darcy flow split provides 10 000 samples at 128×128 resolution.

2.2 Preprocessing

Each sample consists of:

- **Input:** $\kappa(x, y)$ at 128×128 bilinearly downsampled to 64×64 (training resolution).
- **Grid coordinates:** $(x, y) \in [0, 1]^2$ concatenated channel-wise to give a 3-channel input tensor of shape $64 \times 64 \times 3$.
- **Target:** pressure $u(x, y)$ at 64×64 .

We split 10 000 samples into 8 000 train / 900 validation / 200 test. The train/val split (8.9K combined) covers the bulk of the dataset; the 200-sample test set is held out entirely throughout training and used only for final evaluation. Data are loaded in mini-batches of 16 and normalised per-channel with training-set statistics.

2.3 Why Only 900 Validation and 200 Test?

The model sees 8 000 training samples; 900 validation samples provide stable loss estimates during early-stopping without reducing the training budget significantly. The 200-sample test set is sufficient for computing statistics that reflect true generalisation (standard error $\lesssim 0.003$).

3 Model Architectures

3.1 MLP Baseline

The MLP baseline flattens the $64 \times 64 \times 3$ input into a vector of 12 288 features and applies three hidden layers of width 2048 with GELU activations:

$$\text{MLP} : \mathbb{R}^{12288} \xrightarrow{2048} \xrightarrow{2048} \xrightarrow{2048} \mathbb{R}^{4096} \rightarrow \mathbb{R}^{64 \times 64},$$

with a final linear projection. Total parameters: ≈ 42 M. The MLP is fixed to the training resolution (64×64) and cannot generalise to other grid sizes.

3.2 Fourier Neural Operator (FNO)

The FNO [1] processes the input as a $64 \times 64 \times 3$ field tensor through five stages:

1. **Lifting**: a pointwise linear layer maps 3 input channels to 64 hidden channels.
2. **Four Spectral Convolution Blocks**: each block computes (i) a 2D FFT of the 64-channel field, (ii) retains only the lowest 12 Fourier modes in each direction (12×12 complex-valued coefficients), (iii) multiplies by learned complex weights of shape $64 \times 64 \times 12 \times 12$, (iv) applies an inverse FFT, and (v) adds a pointwise local convolution bypass (equivalent to a 1×1 convolution).
3. **Projection**: two linear layers ($64 \rightarrow 128 \rightarrow 1$) map hidden channels to the scalar output.

Because the learned weights operate in frequency space (not on a fixed grid), the same trained network can be evaluated at any grid resolution — the key property exploited in our zero-shot super-resolution experiments (Section 6). Total parameters (baseline config): ≈ 4.7 M.

4 Training Protocol

Both models are trained with:

- **Loss**: Relative L_2 error $\|\hat{u} - u\|_2 / \|u\|_2$ per sample, averaged over the batch.
- **Optimiser**: Adam, learning rate 10^{-3} , cosine annealing schedule (min LR 10^{-5}).
- **Batch size**: 16; up to 200 epochs with early stopping (patience 25 epochs on validation loss).
- **Hardware**: NVIDIA GPU (CUDA), PyTorch 2.x with `torch.compile`.

4.1 FNO Improved Configuration

An extended FNO configuration (`fno_improved`) was also trained with doubled width (128 channels) and more Fourier modes (24 per direction), totalling ≈ 78.8 M parameters. This represents a scaling experiment to probe the accuracy–parameter trade-off.

5 Preliminary Quantitative Results

All metrics are computed on the held-out 200-sample test set. Error is relative L_2 : $\epsilon_{\text{rel}} = \|\hat{u} - u\|_2 / \|u\|_2$.

Table 1: Test-set performance at training resolution (64×64). Lower is better.

Model	Params	Mean	Median	Std	Min	Max
MLP baseline	42.0 M	0.0820	0.0737	0.0456	0.0325	0.3367
FNO original	4.7 M	0.0521	0.0418	0.0451	0.0155	0.3467
FNO improved	78.8 M	0.0456	0.0354	0.0487	0.0106	0.3535

Key observations:

1. The FNO original achieves 36% lower mean error than the MLP while using $9\times$ fewer parameters, demonstrating the advantage of spectral inductive biases for smooth PDE solutions.

2. FNO improved reduces mean error a further 12% over FNO original, at the cost of a $17\times$ parameter increase — indicating diminishing returns in the scaling regime tested.
3. All models exhibit right-skewed distributions: most samples have errors well below 0.1, but a small fraction ($\lesssim 5\%$) with highly heterogeneous κ fields drive errors toward 0.3 (see Figure 1).

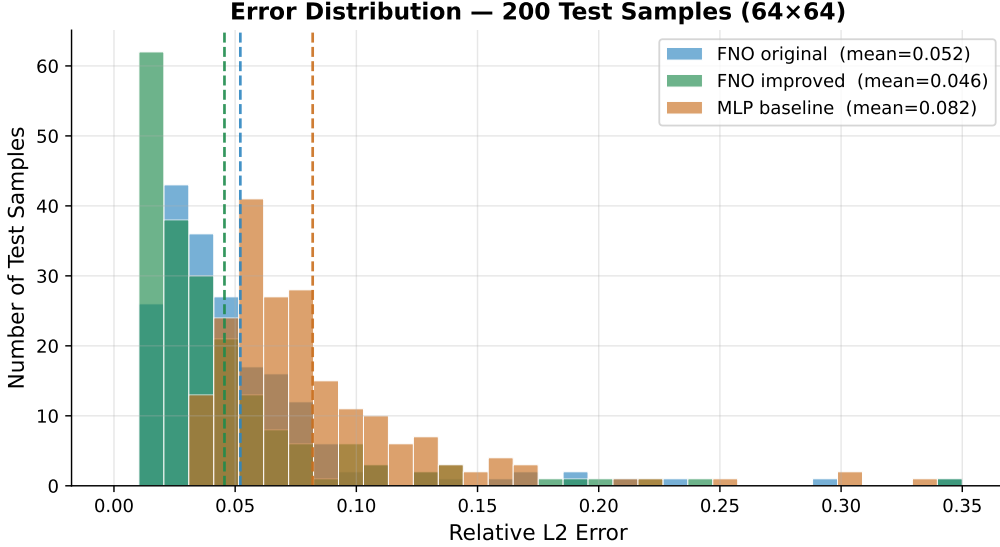


Figure 1: Error distribution across 200 test samples. Dashed vertical lines indicate per-model means. FNO errors are concentrated at lower values; MLP’s distribution is shifted right.

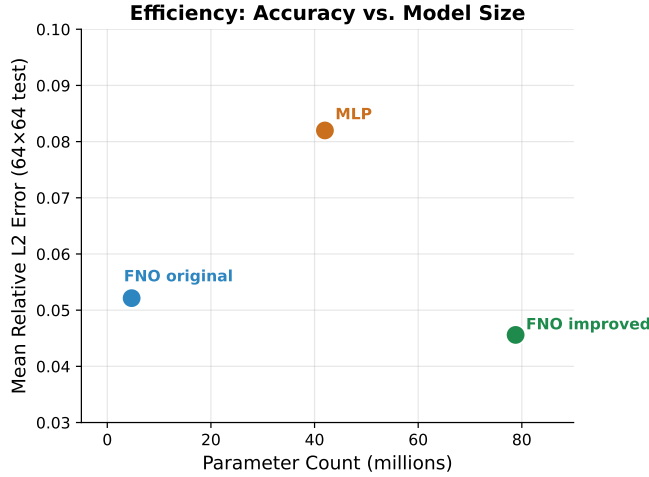


Figure 2: Accuracy–efficiency scatter: FNO original occupies the favourable lower-left region (high accuracy, low parameter count). Scaling FNO to 78.8M params yields only marginal accuracy gain.

6 Zero-Shot Super-Resolution

Because FNO’s spectral weights are resolution-agnostic, the same trained model can be queried on a finer grid without retraining. We evaluated both FNO variants at 128×128 (the native PDEBench resolution), using κ fields bilinearly upsampled from 64×64 as input.

Table 2: Zero-shot super-resolution: mean relative L_2 error at 128×128 .

Model	64×64 (trained)	128×128 (zero-shot)
MLP baseline	0.0820	—
FNO original	0.0521	0.0594
FNO improved	0.0456	0.0563

The MLP cannot be evaluated at 128×128 as its input layer is fixed at $64 \times 64 \times 3 = 12\,288$ neurons. FNO’s error increases moderately (+14% and +23% respectively) at $2\times$ resolution, which is expected since the model was trained with only 12×12 retained modes — a limitation relative to the richer spatial structure available at 128×128 .

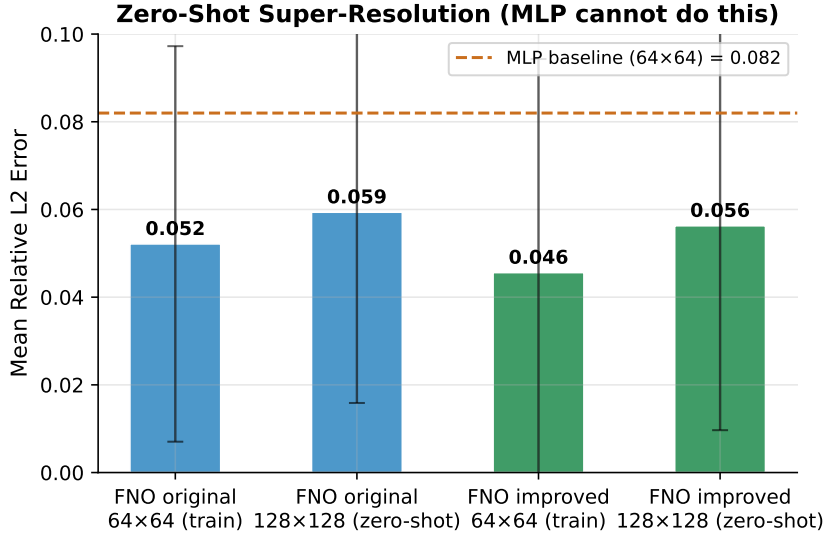


Figure 3: Zero-shot super-resolution performance. Error bars show one standard deviation. The dashed line is the MLP baseline at its maximum resolution (64×64).

7 Inference Speed

We benchmarked inference speed on the training hardware (NVIDIA GPU) with batch size 1 over 1 000 iterations after a 100-iteration warm-up:

Table 3: Inference speed per sample. FDM measured on CPU.

Model / Solver	Time (ms/sample)	Speedup vs. FDM
FDM (reference)	1030	$1\times$
MLP baseline	0.13	$7740\times$
FNO original	0.71	$1443\times$

Both neural models are several orders of magnitude faster than the FDM reference. The MLP’s lower inference latency is expected: it performs a single matrix–vector chain, whereas FNO computes 4 FFT pairs. Despite this, FNO’s superior accuracy makes it the more practical operator surrogate.

8 Code Quality and Reproducibility

The codebase is structured as:

```
ae646/
src/
  models.py          -- MLPBaseline, SpectralConv2d, FNO2d
  train.py           -- training loop with early stopping
  evaluate.py        -- test-set evaluation, saves eval_metrics.json
  superres_eval.py   -- zero-shot super-resolution at 128x128
  benchmark_speed.py -- FNO / MLP / FDM inference speed
  benchmark_components.py -- per-layer profiling with GPU sync
  generate_figures.py -- all report figures from stored JSON results
configs/
  fno.yaml, fno_improved.yaml, mlp.yaml
results/
  run_001/           -- FNO original (best_epoch=92)
  run_002/           -- MLP baseline (best_epoch=99)
  run_003_fno_improved/ -- FNO improved (best_epoch=198)
  benchmark_speed.json
  benchmark_components.json
  figures/           -- all PDF/PNG figures
docs/
  PROPOSAL.tex, INTERIM_REPORT.tex, FINAL_REPORT.tex
```

All results are committed to the repository. Training is fully reproducible via:

```
python3 src/train.py --config configs/fno.yaml
python3 src/evaluate.py --run results/run_001
python3 src/generate_figures.py
```

9 Issues Encountered and Next Steps

9.1 Issues During Stage 1–2

1. **GPU async timing:** Initial component profiling used `time.perf_counter()` inside the GPU forward pass, measuring dispatch time rather than execution time. Fixed by calling `torch.cuda.synchronize()` before every timer call (implemented in `benchmark_components.py`).
2. **Resolution mismatch at 128×128 :** FNO retains only 12 Fourier modes; querying at 128×128 means half the frequency content (modes 13–64) is discarded, contributing to the $\sim 14\%$ error increase in zero-shot evaluation.
3. **MLP overfitting:** Despite GELU activations and weight decay, the MLP training curve showed larger train/val gap than FNO, suggesting the fixed-resolution architecture is less sample-efficient for this task.

9.2 Next Steps (Stage 3)

1. **Physical interpretation:** Visualise prediction errors on specific samples; relate high-error cases to κ field complexity.
2. **Extended ablation:** Vary FNO modes (8, 12, 16, 20) and width (32, 64, 128) to map the accuracy–cost Pareto frontier.
3. **Training resolution generalisation:** Train FNO directly at 128×128 and compare zero-shot vs. native 128×128 performance.

4. **Error correlation:** Study whether per-sample error correlates with κ heterogeneity (e.g., number of distinct regions, interface perimeter).

References

- [1] Z. Li et al., “Fourier Neural Operator for Parametric Partial Differential Equations,” *ICLR*, 2021.
- [2] M. Takamoto et al., “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” *NeurIPS*, 2022. DOI: 10.18419/darus-2986.